claude-windows / c1fcb4aa-537d-41fd-858e-53f93349bf5a

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\subagents\workflows\wf_41c8dc8a-8a4\agent-ab1c40219df374802.jsonl`
- SHA-256: `3c9a1ef8f89967fb3c0dcc611bfc6218024e777839b9f258127f85c96c9bff09`
- Source modified: `2026-07-02T03:02:27+00:00`
- Imported at: `2026-07-05T16:48:24+00:00`
- project: `wf_41c8dc8a-8a4`
- session_id: `c1fcb4aa-537d-41fd-858e-53f93349bf5a`

Transcript

1. user (2026-07-02T03:00:07.022Z)

```
You are an adversarial verifier in a tech-debt audit. Your default stance: REFUTE. Today is
2026-07-02.
A finder claims the following about the repo at C:/Users/avidu/Projects/khelsutra-
guru/badminton-highlight-indexer:

{
  "title": "upload_dir default 'output/uploads' is CWD-relative and never app-home-resolved ?
packaged --app writes uploads into an arbitrary CWD (or 500s on an unwritable one) while
DB/reels live under RALLY_APP_HOME",
  "category": "critical-fix",
  "severity": "P2",
  "file": "backend/api/uploads.py",
  "line": 50,
  "evidence": "uploads.py:50 `upload_dir = getattr(sec, 'upload_dir', None) or 'output/uploads'`
used verbatim at :89 (os.makedirs), :95, :170 ? grep confirms no resolve_output_dir/app_path is
ever applied to it (only output.output_dir is resolved, main.py:1992 via app_home.py:83-100).
models.py:336 default 'output/uploads'.",
  "why_it_matters": "Breaks or misplaces the primary ingest path of the packaged app; exactly
the class of CWD-vs-app-home bug P0a was built to eliminate (app_home.py:1-8).",
  "proposed_fix": "Resolve upload_dir through resolve_output_dir()/app_path() in _upload_cfg
(relative ? app-home-rooted; absolute passes through), mirroring output_dir.",
  "effort": "S",
  "tracked_in": ""
}

Repo root candidates: C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer,
C:/Users/avidu/Projects/khelsutra-guru/khelsutra, C:/Users/avidu/Projects/khelsutra-guru/sports-
data-collector, C:/Users/avidu/Projects/khelsutra-guru/rally-annotator,
C:/Users/avidu/Projects/khelsutra-guru/sports-obsreport (pick from the finding's repo field;
non-git dirs: 'C:/Users/avidu/Projects/khelsutra-guru/Annotation Setup',
C:/Users/avidu/Projects/khelsutra-guru/khelsutra-screencast).
VERIFY against the CURRENT working tree (just pulled to origin master/main): (1) does the cited
code actually exist and behave as claimed at file:line? (2) was it already fixed by a recent
commit (git log --oneline -15 -- <file>)? (3) is the severity honest - construct the concrete
failure scenario or downgrade; (4) is the proposed fix sane and correctly scoped; (5) is it
owner-gated per: alpha/serving go-live, Studio P10/P11 corpus-promotion/training,
product/strategy/licensing, real cloud spend, counsel ToS/DPA?
HARD RULES: READ-ONLY (no file modifications, no installs, no test-suite runs). gh read-only
commands allowed. Your reasoning MUST cite file:line you actually read. CONFIRMED requires you
personally saw the defect/debt in current code; if you cannot reproduce the claim from the code,
say REFUTED; if a commit fixed it, ALREADY_FIXED.
```

2. user (2026-07-02T03:00:13.425Z)

```
    """Chunked upload (B2, PR-2) ? extracted from backend.main into its own router.

    Browser uploads arrive in sequential parts ? security.max_part_mb (free-tier edge
    proxies cap request bodies at ~100 MB ? HOSTING_PLAN §2), are assembled server-side
    under security.upload_dir, then submitted to /api/process by the returned path.
    Upload state is in-process by design (this is the single-box alpha): a server restart
    aborts partial uploads and the client re-inits. Per-user partitioning of upload_dir
    arrives with PR-3 identity scoping.

    The startup config lives on ``backend.main.global_config`` (set ONLY in ``main()``).
    ``_upload_cfg`` resolves it through the ``backend.main`` module object so it always
    reads the live value the server set at startup (and so tests that monkeypatch
    ``backend.main.global_config`` flow through unchanged).
    """

    import os
    import threading
    import uuid
    from typing import Any, Dict, Optional

    from fastapi import APIRouter, HTTPException, Request
    from pydantic import BaseModel

    from backend.utils.freespace import require_free_bytes  # HTTP-507 disk guard (P2, D11)
    from backend.utils.hashing import compute_video_id  # canonical file-identity hashing
    from backend.utils.paths import safe_output_filename

    router = APIRouter()

    _uploads: Dict[str, Dict[str, Any]] = {}
    _uploads_lock = threading.Lock()


    class UploadInitRequest(BaseModel):
        filename: str
        total_size_bytes: Optional[int] = None


    class UploadCompleteRequest(BaseModel):
        total_parts: int


    def _upload_cfg():
        # Resolved lazily (import inside the function) to read the LIVE startup config that
        # main() sets on the module global, and to avoid a circular import at module load
        # (main.py imports this router; this module must not import main at import time).
        import backend.main as _main

        sec = getattr(_main.global_config, "security", None)
        upload_dir = getattr(sec, "upload_dir", None) or "output/uploads"
        max_bytes = int(getattr(sec, "max_upload_mb", 4096) or 4096) * 1024 * 1024
        part_bytes = int(getattr(sec, "max_part_mb", 95) or 95) * 1024 * 1024
        exts = [
            e.lower().lstrip(".")
            for e in (getattr(sec, "allowed_upload_extensions", None) or ["mp4", "mov"])
        ]
        return upload_dir, max_bytes, part_bytes, exts


    def _abort_upload(upload_id: str) -> None:
        with _uploads_lock:
            st = _uploads.pop(upload_id, None)
        if st:
            try:
                os.remove(st["tmp_path"])
```

```
            except OSError:
                pass


    @router.post("/api/upload/init")
    def upload_init(req: UploadInitRequest):
        """Start a chunked upload. Returns upload_id; PUT sequential parts next."""
        upload_dir, max_bytes, part_bytes, exts = _upload_cfg()
        try:
            name = safe_output_filename(req.filename)
        except ValueError as e:
            raise HTTPException(status_code=400, detail=str(e)) from e
        ext = os.path.splitext(name)[1].lower().lstrip(".")
        if ext not in exts:
            raise HTTPException(
                status_code=400,
                detail=f"Extension '.{ext}' not allowed. Allowed: {sorted(exts)}",
            )
        if req.total_size_bytes is not None and req.total_size_bytes > max_bytes:
            raise HTTPException(
                status_code=413,
                detail=f"File exceeds the {max_bytes // (1024 * 1024)} MB upload limit.",
            )
        os.makedirs(upload_dir, exist_ok=True)
        # P2 (D11): fail fast with 507 if the box can't hold the declared upload. Best-
effort ? an
        # undeclared size (total_size_bytes=None) or an unreadable disk skips the guard
(never blocks a
        # legit upload); the per-part total cap in upload_part is the hard backstop either
way.
        require_free_bytes(upload_dir, req.total_size_bytes, stage="upload")
        upload_id = uuid.uuid4().hex
        tmp_path = os.path.join(upload_dir, f".partial-{upload_id}")
        open(tmp_path, "wb").close()
        with _uploads_lock:
            _uploads[upload_id] = {
                "filename": name,
                "tmp_path": tmp_path,
                "parts": 0,
                "bytes": 0,
            }
        return {
            "upload_id": upload_id,
            "max_part_mb": part_bytes // (1024 * 1024),
            "next_part": f"/api/upload/{upload_id}/part/0",
        }


    @router.put("/api/upload/{upload_id}/part/{index}")
    async def upload_part(upload_id: str, index: int, request: Request):
        """Append one part (raw bytes body). Parts are strictly sequential (0,1,2,...)."""
        _, max_bytes, part_bytes, _ = _upload_cfg()
        with _uploads_lock:
            st = _uploads.get(upload_id)
        if st is None:
            raise HTTPException(status_code=404, detail=f"Unknown upload_id '{upload_id}'")
        if index != st["parts"]:
            raise HTTPException(
                status_code=409,
                detail=f"Out-of-order part: expected {st['parts']}, got {index} (parts are
sequential)",
            )

        received = 0
        overflow = None
```

```
        with open(st["tmp_path"], "ab") as fh:
            async for chunk in request.stream():
                received += len(chunk)
                if received > part_bytes:
                    overflow = (
                        f"Part exceeds the {part_bytes // (1024 * 1024)} MB per-part limit."
                    )
                    break
                if st["bytes"] + received > max_bytes:
                    overflow = (
                        f"Upload exceeds the {max_bytes // (1024 * 1024)} MB total limit."
                    )
                    break
                fh.write(chunk)
        if overflow:
            _abort_upload(upload_id)
            raise HTTPException(
                status_code=413, detail=overflow + " Upload aborted ? re-init to retry."
            )

        with _uploads_lock:
            st["parts"] += 1
            st["bytes"] += received
        return {
            "upload_id": upload_id,
            "received_parts": st["parts"],
            "received_bytes": st["bytes"],
        }


    @router.post("/api/upload/{upload_id}/complete")
    def upload_complete(upload_id: str, req: UploadCompleteRequest):
        """Finalize: verify part count, move into place, return {path, video_id} for
/api/process."""
        upload_dir, _, _, _ = _upload_cfg()
        with _uploads_lock:
            st = _uploads.get(upload_id)
        if st is None:
            raise HTTPException(status_code=404, detail=f"Unknown upload_id '{upload_id}'")
        if st["parts"] == 0 or st["parts"] != req.total_parts:
            raise HTTPException(
                status_code=409,
                detail=f"Expected {req.total_parts} part(s), received {st['parts']}.",
            )
        final_path = os.path.join(upload_dir, st["filename"])
        if os.path.exists(
            final_path
        ):  # never overwrite an earlier upload with the same name
            stem, ext = os.path.splitext(st["filename"])
            final_path = os.path.join(upload_dir, f"{stem}-{upload_id[:8]}{ext}")
        os.replace(st["tmp_path"], final_path)
        with _uploads_lock:
            _uploads.pop(upload_id, None)
        video_id = compute_video_id(final_path)
        return {
            "path": os.path.abspath(final_path),
            "video_id": video_id,
            "size_bytes": os.path.getsize(final_path),
            "next": "POST /api/process with this path",
        }


    @router.delete("/api/upload/{upload_id}")
    def upload_abort(upload_id: str):
        """Abort an in-flight upload and delete its partial file."""
```

```
191         with _uploads_lock:
192             known = upload_id in _uploads
193         if not known:
194             raise HTTPException(status_code=404, detail=f"Unknown upload_id '{upload_id}'")
195         _abort_upload(upload_id)
196         return {"status": "aborted", "upload_id": upload_id}
197
```

3. user (2026-07-02T03:00:15.095Z)

```
f28af73 P2: engine free-space guard (HTTP 507 + WSL frame-stage BLOCK) (#407)
b6214e5 chore: ruff format the whole repo (259 files) (#387)
e67add1 refactor(api): extract chunked-upload island from main into backend.api.uploads router
(#228)
e410136 api: /api/videos surfaces fps + resolution (from the stored ffprobe details) (#413)
81a78bf api: /api/videos returns video_id + a friendly match_name (fix disappearing name) (#412)
8b16cf4 docs: reflect today's storage endpoints + the corpus/user-video boundary (#411)
```

4. user (2026-07-02T03:00:29.463Z)

```
badminton-highlight-indexer\tools\cleanup_caches.py:5:(``security.upload_dir``, default
``output/uploads``) ? classifies each entry as
badminton-highlight-indexer\tools\cleanup_caches.py:30:automatically when ``wsl`` is
unavailable). ``--uploads-dir`` must match ``security.upload_dir``
badminton-highlight-indexer\tools\cleanup_caches.py:56:        # The upload landing zone
(security.upload_dir). Uploaded source videos and the
badminton-highlight-indexer\tools\cleanup_caches.py:274:    Flat by design today (uploads.py
writes ``<upload_dir>/<filename>`` plus
badminton-highlight-indexer\tools\cleanup_caches.py:403:        help="upload landing zone to
sweep (default <output-dir>/uploads; match security.upload_dir)",
badminton-highlight-indexer\backend\api\uploads.py:5:under security.upload_dir, then submitted
to /api/process by the returned path.
badminton-highlight-indexer\backend\api\uploads.py:7:aborts partial uploads and the client re-
inits. Per-user partitioning of upload_dir
badminton-highlight-indexer\backend\api\uploads.py:50:    upload_dir = getattr(sec,
"upload_dir", None) or "output/uploads"
badminton-highlight-indexer\backend\api\uploads.py:57:    return upload_dir, max_bytes,
part_bytes, exts
badminton-highlight-indexer\backend\api\uploads.py:73:    upload_dir, max_bytes, part_bytes,
exts = _upload_cfg()
badminton-highlight-indexer\backend\api\uploads.py:89:    os.makedirs(upload_dir, exist_ok=True)
badminton-highlight-indexer\backend\api\uploads.py:93:    require_free_bytes(upload_dir,
req.total_size_bytes, stage="upload")
badminton-highlight-indexer\backend\api\uploads.py:95:    tmp_path = os.path.join(upload_dir,
f".partial-{upload_id}")
badminton-highlight-indexer\backend\api\uploads.py:160:    upload_dir, _, _, _ = _upload_cfg()
badminton-highlight-indexer\backend\api\uploads.py:170:    final_path = os.path.join(upload_dir,
st["filename"])
badminton-highlight-indexer\backend\api\uploads.py:175:        final_path =
os.path.join(upload_dir, f"{stem}-{upload_id[:8]}{ext}")
badminton-highlight-indexer\backend\config\models.py:335:    # contain upload_dir or
/api/process will reject the uploaded paths.
badminton-highlight-indexer\backend\config\models.py:336:    upload_dir: str = "output/uploads"
badminton-highlight-indexer\backend\utils\freespace.py:4:``upload_dir``), a **fetch-to-scratch**
of a bucket/Drive source (a full local copy under the temp
badminton-highlight-indexer\backend\utils\freespace.py:32:    ancestor (an
``upload_dir``/scratch target may not exist yet at guard time)."""
badminton-highlight-indexer\tests\test_upload_download.py:31:    cfg.security.upload_dir =
str(tmp_path / "up")
badminton-highlight-indexer\tests\test_upload_download.py:125:        f for f in
os.listdir(cfg.security.upload_dir) if f.startswith(".partial-")
badminton-highlight-indexer\tests\test_upload_download.py:206:        f for f in
os.listdir(cfg.security.upload_dir) if f.startswith(".partial-")
badminton-highlight-indexer\tests\test_upload_download.py:232:        f for f in
os.listdir(cfg.security.upload_dir) if f.startswith(".partial-")
```

5. user (2026-07-02T03:00:29.550Z)

```
1       """Resolved application-data HOME for a packaged, batteries-included install.
2
3       PACKAGING_PLAN.md P0a. Today every piece of mutable state ? ``config.json``, ``.env``,
4       the SQLite DB, ``output/`` ? is resolved RELATIVE TO THE CURRENT WORKING DIRECTORY, and
5       ``scripts/doctor.py`` writes ``config.json`` to the repo root while the server reads it
from
6       the CWD. Launched from an OS app menu (CWD ? repo root) those disagree and the user
silently
7       "loses" their index. This module introduces ONE resolver every state path roots on.
8
9       Design contract ? **DEFAULT IS BYTE-IDENTICAL TO TODAY**:
10        * ``RALLY_APP_HOME`` env set  ? that directory is the app home (the packaged launcher
sets it).
11        * ``RALLY_APP_HOME`` unset    ? ``Path(".")`` i.e. the CWD, exactly the pre-P0a
behaviour.
12
13      The OS-default location (``%APPDATA%`` / ``~/Library/Application Support`` /
``~/.local/share``)
14      is computed by :func:`default_os_app_home` but is **NOT** auto-selected here ? the
launcher (P2)
15      calls it and exports ``RALLY_APP_HOME`` explicitly. Keeping the resolver opt-in means
importing
16      this module has zero behavioural effect on the repo/dev/CI/test paths (no surprise
writes into a
17      real user profile during a test run).
18
19      stdlib-only (os, platform, pathlib) so it is safe to import at ``backend.main`` line 1 ?
before
20      the heavier pydantic-backed ``backend.config`` package ? keeping the early
``load_dotenv`` path light.
21      """
22
23      from __future__ import annotations
24
25      import os
26      import platform
27      from pathlib import Path
28
29      #: Environment variable the packaged launcher sets to pin the app-data home.
30      APP_HOME_ENV = "RALLY_APP_HOME"
31
32      #: OS-app-dir leaf used by :func:`default_os_app_home` (the product brand).
33      APP_DIR_NAME = "Khelsutra"
34
35
36      def app_home() -> Path:
37          """The resolved application-data home.
38
39          ``RALLY_APP_HOME`` (``~`` expanded) when set; otherwise ``Path(".")`` ? the CWD,
byte-identical
40          to the pre-P0a behaviour. Does NOT create the directory (callers create on write).
41          """
42          env = os.environ.get(APP_HOME_ENV)
43          if env and env.strip():
44              return Path(env.strip()).expanduser()
45          return Path(".")
46
47
48      def app_path(*parts: str | os.PathLike[str]) -> Path:
49          """A path under :func:`app_home` (e.g. ``app_path("output")``)."""
50          return app_home().joinpath(*[str(p) for p in parts])
51
52
```

```
    def default_config_path() -> Path:
        """``<app-home>/config.json`` ? the config file the loader reads and ``--setup``
writes.

        With ``RALLY_APP_HOME`` unset this is ``Path("config.json")`` (CWD-relative),
unchanged.
        """
        return app_home() / "config.json"


    def default_env_path() -> Path:
        """``<app-home>/.env`` ? the dotenv file the server prefers (e.g.
``GEMINI_API_KEY``)."""
        return app_home() / ".env"


    def env_file_to_load() -> Path | None:
        """The ``.env`` the server should load, or ``None`` meaning "use python-dotenv's
DEFAULT
        (frame-relative walk-up) search" ? exactly the pre-P0a bare ``load_dotenv()``
behaviour.

        Returns the app-home ``.env`` ONLY when ``RALLY_APP_HOME`` is set AND that file
exists. With the
        env UNSET this returns ``None`` so the caller keeps the historical search and does
NOT newly
        prefer a stray CWD ``.env`` (bare ``load_dotenv()`` walks up from the caller file,
not the CWD).
        """
        home = app_home()
        if (
            str(home) == "."
        ):  # env unset (or pinned to CWD) ? historical load_dotenv() search
            return None
        candidate = home / ".env"
        return candidate if candidate.exists() else None


    def resolve_output_dir(output_dir: str) -> str:
        """Root a RELATIVE ``output_dir`` under :func:`app_home`; pass an ABSOLUTE one
through.

        With ``RALLY_APP_HOME`` UNSET the input is returned VERBATIM (the default
``"output"`` when
        empty) ? byte-identical to the pre-P0a code that stored ``--output-dir`` as-is (no
pathlib
        normalisation of ``./x`` / separators / trailing slash). With it SET, a relative dir
like
        ``"output"`` becomes ``<app-home>/output`` so the SQLite DB and reels live with the
rest of the
        app's state; an absolute path ? or a Windows drive-anchored one (``C:\\x`` /
``C:x``) ? passes
        through untouched.
        """
        home = app_home()
        if str(home) == ".":  # env unset (or pinned to CWD) ? verbatim, today's behaviour
            return output_dir or "output"
        if not output_dir:
            return str(home / "output")
        if os.path.isabs(output_dir) or os.path.splitdrive(output_dir)[0]:
            return output_dir
        return str(home / output_dir)


    def default_os_app_home(app_name: str = APP_DIR_NAME) -> Path:
```

```
104         """The OS-conventional per-user app-data dir (for the packaged LAUNCHER to export as
105         ``RALLY_APP_HOME`` ? NOT auto-selected by :func:`app_home`).
106
107         * Windows ? ``%APPDATA%\\<app_name>`` (``~/AppData/Roaming/<app_name>`` if unset)
108         * macOS   ? ``~/Library/Application Support/<app_name>``
109         * Linux/other ? ``$XDG_DATA_HOME/<app_name>`` (``~/.local/share/<app_name>`` if
unset)
110         """
111         system = platform.system()
112         if system == "Windows":
113             base = os.environ.get("APPDATA") or str(Path.home() / "AppData" / "Roaming")
114         elif system == "Darwin":
115             base = str(Path.home() / "Library" / "Application Support")
116         else:
117             base = os.environ.get("XDG_DATA_HOME") or str(Path.home() / ".local" / "share")
118         return Path(base) / app_name
119
```

6. user (2026-07-02T03:00:39.096Z)

```
badminton-highlight-indexer\backend\main.py:25:from backend.utils.app_home import
env_file_to_load, resolve_output_dir
badminton-highlight-indexer\backend\main.py:1927:            out_dir = resolve_output_dir(
badminton-highlight-indexer\backend\main.py:1991:    # unset, resolve_output_dir("output") ==
"output" (CWD-relative) ? byte-identical to before.
badminton-highlight-indexer\backend\main.py:1992:    global_config.output.output_dir =
resolve_output_dir(args.output_dir)
badminton-highlight-indexer\backend\utils\app_home.py:36:def app_home() -> Path:
badminton-highlight-indexer\backend\utils\app_home.py:48:def app_path(*parts: str |
os.PathLike[str]) -> Path:
badminton-highlight-indexer\backend\utils\app_home.py:49:    """A path under :func:`app_home`
(e.g. ``app_path("output")``)."""
badminton-highlight-indexer\backend\utils\app_home.py:50:    return app_home().joinpath(*[str(p)
for p in parts])
badminton-highlight-indexer\backend\utils\app_home.py:58:    return app_home() / "config.json"
badminton-highlight-indexer\backend\utils\app_home.py:63:    return app_home() / ".env"
badminton-highlight-indexer\backend\utils\app_home.py:74:    home = app_home()
badminton-highlight-indexer\backend\utils\app_home.py:83:def resolve_output_dir(output_dir: str)
-> str:
badminton-highlight-indexer\backend\utils\app_home.py:93:    home = app_home()
badminton-highlight-indexer\backend\utils\app_home.py:103:def default_os_app_home(app_name: str
= APP_DIR_NAME) -> Path:
```

7. user (2026-07-02T03:00:39.789Z)

```
320         # Third-party HTTP/RPC client loggers (httpx, httpcore, urllib3, google-genai,
321         # google.auth, grpc) emit one INFO line per request ? dozens per Gemini run, which
322         # buries our own [rally]/[compile] lines in run.log during manual QA. Default raises
323         # them to WARNING; set true to restore full chatter for request-flow debugging.
324         verbose_http: bool = False
325
326
327     class SecurityConfig(BaseModel):
328         # When set, /api/process video_path must resolve under this directory (hosted/tenant
329         # mode). Default None preserves the single-user local 'any local file' workflow.
330         allowed_video_root: Optional[str] = None
331
332         # --- Chunked upload (B2, PR-2) ---
333         # Landing zone for browser uploads (assembled from parts). Per-user subdirectories
334         # arrive with PR-3 identity scoping. ? In hosted mode, set allowed_video_root to
335         # contain upload_dir or /api/process will reject the uploaded paths.
336         upload_dir: str = "output/uploads"
337         # Whole-file cap (default 4 GB) and per-part cap. Parts stay under ~100 MB because
338         # free-tier edge proxies cap request bodies there (HOSTING_PLAN §2).
339         max_upload_mb: int = 4096
```

```
340         max_part_mb: int = 95
341         allowed_upload_extensions: List[str] = ["mp4", "mov"]
342
343         # --- M9 edge auth (Cloudflare Access, D9) ---
344         # DEFAULT-OFF: edge_auth_enabled=False ? no JWT is required/verified (the local
single-user
345         # path is unchanged). When True, /api/process requires + verifies the `Cf-Access-
Jwt-Assertion`
346         # header against the team JWKS (so a direct hit to the Cloud Run URL can't spoof the
identity
347         # header) and tags the job with the verified user_id (owner_id). The team domain
(e.g.
348         # ``https://<team>.cloudflareaccess.com``) supplies the JWKS + issuer; the AUD is
the CF Access
349         # application's audience tag. NO secrets here ? these are public identifiers.
```

8. assistant (2026-07-02T03:00:54.465Z)

The core claim is verified. Let me check whether a packaged launcher exists and whether it sets CWD to app home (which would mitigate severity).

9. user (2026-07-02T03:01:00.700Z)

```
badminton-highlight-indexer\scripts\install.py:8:  * creates a per-OS launcher shortcut that
pins ``RALLY_APP_HOME`` to the OS app-data dir and runs
badminton-highlight-indexer\scripts\install.py:72:    The shortcut pins ``RALLY_APP_HOME`` then
runs ``indexer --app``. PURE (no IO) so it's testable.
badminton-highlight-indexer\scripts\install.py:81:            f'set "RALLY_APP_HOME={home}"\r\n'
badminton-highlight-indexer\scripts\install.py:89:            f'export
RALLY_APP_HOME="{home}"\n'
badminton-highlight-indexer\scripts\install.py:97:        f'export RALLY_APP_HOME="{home}"\n'
badminton-highlight-indexer\tests\test_app_home.py:3:The load-bearing guarantee: with
``RALLY_APP_HOME`` UNSET, every resolver is byte-identical to the
badminton-highlight-indexer\tests\test_app_home.py:18:    """Each test starts with
RALLY_APP_HOME unset unless it sets it explicitly."""
badminton-highlight-indexer\tests\test_app_home.py:54:# ---- RALLY_APP_HOME set ? everything
roots there ----
badminton-highlight-indexer\tests\test_app_home.py:163:# ---- env_file_to_load: app-home .env
preferred ONLY when RALLY_APP_HOME is set ----
badminton-highlight-indexer\scripts\doctor.py:8:# NO LONGER written here ? it follows
backend.utils.app_home.default_config_path() (RALLY_APP_HOME,
badminton-highlight-indexer\scripts\doctor.py:74:    # (RALLY_APP_HOME, else CWD/config.json) ?
previously this wrote to the repo root while
badminton-highlight-indexer\tests\test_app_launcher.py:178:        "RALLY_APP_HOME": str(
badminton-highlight-indexer\tests\test_app_launcher.py:206:        assert seeded, "config.json
was not seeded under RALLY_APP_HOME"
badminton-highlight-indexer\backend\config\loader.py:33:# ``RALLY_APP_HOME`` (PACKAGING_PLAN.md
P0a) and equals ``Path("config.json")`` (CWD-relative) when
badminton-highlight-indexer\backend\config\loader.py:36:# from a later RALLY_APP_HOME.)
badminton-highlight-indexer\tests\test_doctor.py:36:        "config.json must land under
RALLY_APP_HOME (the server's read path)"
badminton-highlight-indexer\backend\main.py:21:# PACKAGING_PLAN.md P0a: prefer an app-home .env
ONLY when RALLY_APP_HOME is set (the packaged
badminton-highlight-indexer\backend\main.py:1967:    # roots under RALLY_APP_HOME when the
packaged shortcut pins it) BEFORE load_app_config reads
badminton-highlight-indexer\backend\main.py:1990:    # P0a: a RELATIVE output dir roots under
the app-home (RALLY_APP_HOME); with the env
badminton-highlight-indexer\backend\main.py:2022:                f"use a different --output-dir
(RALLY_APP_HOME) to run an independent instance."
badminton-highlight-indexer\backend\utils\app_home.py:10:  * ``RALLY_APP_HOME`` env set  ? that
directory is the app home (the packaged launcher sets it).
badminton-highlight-indexer\backend\utils\app_home.py:11:  * ``RALLY_APP_HOME`` unset    ?
``Path(".")`` i.e. the CWD, exactly the pre-P0a behaviour.
badminton-highlight-indexer\backend\utils\app_home.py:15:calls it and exports ``RALLY_APP_HOME``
explicitly. Keeping the resolver opt-in means importing
```

```
badminton-highlight-indexer\backend\utils\app_home.py:30:APP_HOME_ENV = "RALLY_APP_HOME"
badminton-highlight-indexer\backend\utils\app_home.py:39:    ``RALLY_APP_HOME`` (``~`` expanded)
when set; otherwise ``Path(".")`` ? the CWD, byte-identical
badminton-highlight-indexer\backend\utils\app_home.py:56:    With ``RALLY_APP_HOME`` unset this
is ``Path("config.json")`` (CWD-relative), unchanged.
badminton-highlight-indexer\backend\utils\app_home.py:70:    Returns the app-home ``.env`` ONLY
when ``RALLY_APP_HOME`` is set AND that file exists. With the
badminton-highlight-indexer\backend\utils\app_home.py:86:    With ``RALLY_APP_HOME`` UNSET the
input is returned VERBATIM (the default ``"output"`` when
badminton-highlight-indexer\backend\utils\app_home.py:105:    ``RALLY_APP_HOME`` ? NOT auto-
selected by :func:`app_home`).
badminton-highlight-indexer\backend\infrastructure\database.py:114:                    f"a
different data directory (RALLY_APP_HOME). DB: {self.db_path}"
badminton-highlight-indexer\tests\test_p2b_install.py:107:    assert "RALLY_APP_HOME" in content
and str(home) in content
badminton-highlight-indexer\tests\test_server_entrypoint.py:118:    """PACKAGING_PLAN.md P0a
end-to-end: with RALLY_APP_HOME set and a RELATIVE --output-dir, the
badminton-highlight-indexer\tests\test_server_entrypoint.py:141:        "RALLY_APP_HOME":
str(home),
badminton-highlight-indexer\tests\test_server_entrypoint.py:165:        "DB was NOT created
under RALLY_APP_HOME ? resolve_output_dir is not wired into main() init.\n"
```

10. user (2026-07-02T03:01:01.509Z)

```
badminton-highlight-indexer\tests\test_doctor.py:52:    monkeypatch.chdir(tmp_path)
badminton-highlight-indexer\tests\test_app_home.py:49:    monkeypatch.chdir(tmp_path)
badminton-highlight-indexer\tests\test_wasb_cache.py:227:        os.chdir(osp.dirname(here) or
here)
badminton-highlight-indexer\tests\test_wasb_cache.py:230:        os.chdir(here)
badminton-highlight-indexer\tests\test_wsl_tilde_quoting.py:88:    monkeypatch.chdir(tmp_path)
```

11. assistant (2026-07-02T03:01:08.565Z)

```
The launcher script matters ? if it `cd`s into the app home before running, CWD-relative would
coincide with app home. Let me read install.py.
```

12. user (2026-07-02T03:01:09.132Z)

```
1       #!/usr/bin/env python3
2       """Batteries-included installer for the KhelSutra highlight indexer (PACKAGING_PLAN.md
P2b).
3
4       Cross-platform (Windows / macOS / Linux), no native component (O1):
5
6         * installs the LIGHT (torch-free) package via ``uv tool install`` (preferred ? an
isolated tool
7           env that puts ``indexer`` on PATH) or ``pip install`` (fallback);
8         * creates a per-OS launcher shortcut that pins ``RALLY_APP_HOME`` to the OS app-data
dir and runs
9           ``indexer --app`` ? so config / .env / DB / reels live in ONE stable place
10          (mirrors ``backend.utils.app_home.default_os_app_home``). Pinning the env BEFORE the
process
11          starts is load-bearing: the engine's ``load_dotenv`` runs at import, so only an env
set by the
12          launcher (not by the engine) roots the ``.env`` in the app-home;
13        * writes an uninstall manifest so ``scripts/uninstall.py`` can cleanly reverse it.
14
15      The truly-local + ``motion`` default config is seeded by the ENGINE itself on the first
16      ``indexer --app`` run (``write_light_default_config``), so it works regardless of
install method;
17      this script only wires the install + the launcher.
18
19      Usage::
20
```

```
        python scripts/install.py                       # install '.' (this checkout), uv-if-present else pip
        python scripts/install.py --target badminton-highlight-indexer   # from PyPI (once published)
        python scripts/install.py --method pip          # force pip
        python scripts/install.py --dry-run             # print the plan, change nothing

    stdlib-only so it runs as a standalone bootstrap BEFORE the package exists.
    """

    from __future__ import annotations

    import argparse
    import json
    import os
    import platform
    import shutil
    import subprocess
    import sys
    from datetime import datetime, timezone
    from pathlib import Path

    PACKAGE = "badminton-highlight-indexer"
    APP_DIR_NAME = "Khelsutra"  # mirrors backend.utils.app_home.APP_DIR_NAME
    MANIFEST_NAME = "uninstall_manifest.json"


    # --- pure helpers (unit-tested) ------------------------------------------------


    def default_os_app_home(
        app_name: str = APP_DIR_NAME,
        system: str | None = None,
        env: dict[str, str] | None = None,
    ) -> Path:
        """The OS-conventional per-user app-data dir ? a standalone mirror of
        ``backend.utils.app_home.default_os_app_home`` (this script runs before the package exists)."""
        system = system or platform.system()
        e = os.environ if env is None else env
        if system == "Windows":
            base = e.get("APPDATA") or str(Path.home() / "AppData" / "Roaming")
        elif system == "Darwin":
            base = str(Path.home() / "Library" / "Application Support")
        else:
            base = e.get("XDG_DATA_HOME") or str(Path.home() / ".local" / "share")
        return Path(base) / app_name


    def shortcut_spec(
        system: str, app_home: Path, indexer_cmd: str = "indexer"
    ) -> tuple[str, str, bool]:
        """Return ``(filename, content, make_executable)`` for the per-OS launcher shortcut.

        The shortcut pins ``RALLY_APP_HOME`` then runs ``indexer --app``. PURE (no IO) so it's testable.
        """
        home = str(app_home)
        # Quote the command: ``indexer_cmd`` may be an ABSOLUTE path (resolved at install time) that can
        # contain spaces; quoting is harmless for the bare ``indexer`` fallback too.
        if system == "Windows":
            content = (
                "@echo off\r\n"
```

```
                "rem KhelSutra launcher (PACKAGING_PLAN.md P2b) ? pins app-home then opens
the app.\r\n"
                f'set "RALLY_APP_HOME={home}"\r\n'
                f'"{indexer_cmd}" --app\r\n'
            )
            return "KhelSutra.bat", content, False
        if system == "Darwin":
            content = (
                "#!/bin/bash\n"
                "# KhelSutra launcher (PACKAGING_PLAN.md P2b) ? pins app-home then opens the
app.\n"
                f'export RALLY_APP_HOME="{home}"\n'
                f'exec "{indexer_cmd}" --app\n'
            )
            return "KhelSutra.command", content, True
        # Linux / other
        content = (
            "#!/bin/bash\n"
            "# KhelSutra launcher (PACKAGING_PLAN.md P2b) ? pins app-home then opens the
app.\n"
            f'export RALLY_APP_HOME="{home}"\n'
            f'exec "{indexer_cmd}" --app\n'
        )
        return "khelsutra.sh", content, True


    def build_manifest(
        *, method: str, app_home: Path, shortcut: Path, created: list[Path], timestamp: str
    ) -> dict:
        """The uninstall manifest (PURE). ``created`` = files THIS installer made (safe to
remove);
        ``user_data`` = paths the engine/user own (kept unless ``uninstall.py --purge``)."""
        return {
            "schema": 1,
            "product": "KhelSutra highlight indexer",
            "package": PACKAGE,
            "install_method": method,  # "uv" | "pip"
            "indexer_command": "indexer",
            "app_home": str(app_home),
            "shortcut": str(shortcut),
            "created": [str(p) for p in created],
            "user_data": [
                str(app_home / "config.json"),
                str(app_home / ".env"),
                str(app_home / "output"),
            ],
            "timestamp": timestamp,
        }


    def resolve_method(requested: str, uv_present: bool) -> str:
        """Pick the installer: explicit beats auto; auto prefers uv when present, else
pip."""
        if requested in ("uv", "pip"):
            return requested
        return "uv" if uv_present else "pip"


    def install_command(method: str, target: str) -> list[str]:
        """The argv for the chosen installer (PURE)."""
        if method == "uv":
            return ["uv", "tool", "install", target]
        return [sys.executable, "-m", "pip", "install", target]


```

```
    # --- IO / orchestration
-----------------------------------------------------------------


    def _run(argv: list[str]) -> None:
        print(f"  $ {' '.join(argv)}")
        subprocess.run(argv, check=True)


    def main(argv: list[str] | None = None) -> int:
        ap = argparse.ArgumentParser(
            description="Install the KhelSutra highlight indexer (LIGHT tier)."
        )
        ap.add_argument(
            "--target",
            default=".",
            help="What to install: '.' (this checkout, default) or a PyPI name/spec.",
        )
        ap.add_argument(
            "--method",
            choices=("auto", "uv", "pip"),
            default="auto",
            help="Installer to use (default: uv if present, else pip).",
        )
        ap.add_argument(
            "--dry-run", action="store_true", help="Print the plan; change nothing."
        )
        args = ap.parse_args(argv)

        system = platform.system()
        app_home = default_os_app_home()
        method = resolve_method(args.method, shutil.which("uv") is not None)
        cmd = install_command(method, args.target)
        fname, _, executable = shortcut_spec(
            system, app_home
        )  # content is recomputed post-install
        shortcut = app_home / fname

        print(f"KhelSutra installer ? {PACKAGE} (LIGHT, torch-free)")
        print(f"  OS         : {system}")
        print(f"  installer  : {method}")
        print(f"  target     : {args.target}")
        print(f"  app-home   : {app_home}")
        print(f"  shortcut   : {shortcut}")
        if args.dry_run:
            print("\n[dry-run] would run:")
            print(f"  $ {' '.join(cmd)}")
            print(f"  write shortcut -> {shortcut}")
            print(f"  write manifest -> {app_home / MANIFEST_NAME}")
            print(f"  copy uninstaller -> {app_home / 'uninstall.py'}")
            return 0

        print("\n[1/3] installing the package?")
        try:
            _run(cmd)
        except (subprocess.CalledProcessError, FileNotFoundError) as e:
            print(f"[ERROR] install failed: {e}")
            if method == "uv":
                print("  (uv not usable? re-run with --method pip)")
            return 1

        # Make `indexer` reachable. `uv tool install` drops it in an isolated tool-bin that
is NOT
        # auto-added to PATH ? nudge it onto PATH for future shells. For THIS run, resolve
the absolute
```

```
        # path so the launcher shortcut works even before PATH is refreshed; warn loudly if we can't.
        if method == "uv":
            try:
                subprocess.run(["uv", "tool", "update-shell"], check=False)
            except FileNotFoundError:
                pass
        indexer_exe = shutil.which(
            "indexer"
        )  # None if the install bin dir isn't on THIS process's PATH
        indexer_cmd = indexer_exe or "indexer"

        print("\n[2/3] writing the launcher shortcut + app-home?")
        app_home.mkdir(parents=True, exist_ok=True)
        _, content, _ = shortcut_spec(
            system, app_home, indexer_cmd
        )  # embed the resolved indexer path
        shortcut.write_text(content, encoding="utf-8")
        if executable:
            shortcut.chmod(0o755)
        # Make the uninstaller available next to the data it removes.
        created: list[Path] = [shortcut]
        here = Path(__file__).resolve().parent
        src_uninstall = here / "uninstall.py"
        dst_uninstall = app_home / "uninstall.py"
        if src_uninstall.exists():
            shutil.copyfile(src_uninstall, dst_uninstall)
            created.append(dst_uninstall)

        print("\n[3/3] writing the uninstall manifest?")
        manifest_path = app_home / MANIFEST_NAME
        created.append(manifest_path)
        manifest = build_manifest(
            method=method,
            app_home=app_home,
            shortcut=shortcut,
            created=created,
            timestamp=datetime.now(timezone.utc).isoformat(timespec="seconds"),
        )
        manifest_path.write_text(json.dumps(manifest, indent=2) + "\n", encoding="utf-8")

        print("\n? Installed. To launch KhelSutra:")
        print(f"   run the launcher  {shortcut}   (or `indexer --app` from a shell)")
        print(
            "   The first run seeds a truly-local 'motion' config (offline, no key/GPU needed)."
        )
        if indexer_exe is None:
            print(
                "   ? `indexer` isn't on this shell's PATH yet ? the launcher uses the bare name. If it"
            )
            print(
                "     fails, reopen your terminal (uv just ran `update-shell`), add the install bin dir to"
            )
            print(
                "     PATH, or launch with `python -m backend.main --app` from the install environment."
            )
        print(
            f'   To uninstall:  python "{dst_uninstall}"   (add --purge to also delete your data)'
        )
        return 0
```

```
261
262
263     if __name__ == "__main__":
264         raise SystemExit(main())
265
```

13. user (2026-07-02T03:01:25.019Z)

```
badminton-highlight-indexer\scripts\install.py:16:``indexer --app`` run
(``write_light_default_config``), so it works regardless of install method;
badminton-highlight-indexer\backend\main.py:57:    write_light_default_config,  # P2b:
batteries-included first-run config seeding
badminton-highlight-indexer\backend\main.py:1972:        seeded_path, seeded =
write_light_default_config()
badminton-highlight-indexer\backend\config\__init__.py:15:    write_light_default_config,
badminton-highlight-indexer\backend\config\__init__.py:48:    "write_light_default_config",
badminton-highlight-indexer\backend\config\loader.py:268:def write_light_default_config(
badminton-highlight-indexer\docs\PACKAGING_PLAN.md:113:[Omitted long matching line]
badminton-highlight-indexer\docs\PACKAGING_PLAN.md:133:8. **O8 ? RESOLVED 2026-06-22:** the
shipped DEFAULT (seeded on the first `indexer --app`) = **profile `truly-local` + detector
`motion` + `use_gpu` false** ? a torch-free, offline, zero-setup experience that never surprise-
bills on Gemini nor returns 0 rallies on WASB-without-a-key. The P2c wizard upgrades to Gemini
once a key is supplied. (Implemented in P2b-install via `write_light_default_config`.)
badminton-highlight-indexer\tests\test_p2b_install.py:4:  * the engine-side
``write_light_default_config`` (gate O8 default = truly-local + motion), incl.
badminton-highlight-indexer\tests\test_p2b_install.py:18:from backend.config import
LIGHT_DEFAULT_CONFIG, load_config, write_light_default_config
badminton-highlight-indexer\tests\test_p2b_install.py:31:# --- engine seeding:
write_light_default_config (gate O8) ---------------------------------
badminton-highlight-indexer\tests\test_p2b_install.py:36:    path, written =
write_light_default_config(cfg)
badminton-highlight-indexer\tests\test_p2b_install.py:51:    path, written =
write_light_default_config(cfg)
badminton-highlight-indexer\tests\test_p2b_install.py:63:    _, written =
write_light_default_config(cfg, overwrite=True)
badminton-highlight-indexer\tests\test_p2b_install.py:72:    write_light_default_config(cfg)
```

14. user (2026-07-02T03:01:30.419Z)

```
220         else:
221             # No profile ? same deep merge so a config.json sub-key overrides its
222             # default while NEW defaults added in the model since the file was written
223             # survive ? no silent loss of new knobs on upgrade (D4).
224             merged = _deep_merge(defaults, file_data)
225             suggestion = (
226                 suggest_profile()
227             )  # cheap, env-only (no torch); SUGGESTS only (D15)
228             if suggestion:
229                 logger.debug(
230                     "[CONFIG] no deployment.profile set (manual knobs). Environment suggests
%r ? "
231                     "set deployment.profile or DEPLOYMENT_PROFILE to apply (auto-detect
never "
232                     "auto-applies ? D15).",
233                     suggestion,
234                 )
235
236         return AppConfig.model_validate(merged)
237
238
239     def save_default_config(path: str | Path | None = None) -> Path:
240         """Write a fresh default config.json.
241
242         Used by scripts/doctor.py diagnostics and the --setup CLI path.
243         Overwrites if the file exists.
```

```
244         """
245         cfg_path = Path(path) if path is not None else default_config_path()
246         cfg = AppConfig()
247         cfg_path.parent.mkdir(parents=True, exist_ok=True)
248         cfg_path.write_text(cfg.model_dump_json(indent=4), encoding="utf-8")
249         return cfg_path
250
251
252     # The shipped batteries-included LIGHT-tier default (PACKAGING_PLAN.md P2b / gate O8).
253     # Deliberately MINIMAL: the `truly-local` PROFILE preset (presets.py) supplies
skip_ai_handoff,
254     # local storage and the compute_target, so this stays the handful of knobs a fresh, non-
technical
255     # user gets ? a torch-free, offline, zero-setup experience:
256     #   * default_segmenter "motion"  ? pure-OpenCV, no torch / GPU / weights / API key
(LIGHT tier);
257     #   * profile "truly-local"        ? no Gemini handoff, local storage ? never surprise-
bills on
258     #                                     Gemini nor returns 0 rallies on WASB-without-a-key
(O8 intent);
259     #   * use_gpu false                ? honest for a LIGHT (no-GPU) box (motion ignores it
either way).
260     # The first-run wizard (P2c) upgrades this to Gemini once the user supplies a key.
261     LIGHT_DEFAULT_CONFIG: dict[str, Any] = {
262         "sport": "badminton",
263         "deployment": {"profile": "truly-local"},
264         "indexing": {"default_segmenter": "motion", "use_gpu": False},
265     }
266
267
268     def write_light_default_config(
269         path: str | Path | None = None, *, overwrite: bool = False
270     ) -> tuple[Path, bool]:
271         """Seed the batteries-included LIGHT default config (``LIGHT_DEFAULT_CONFIG``).
272
273         Returns ``(path, written)``. Unlike :func:`save_default_config` (which dumps a FULL
AppConfig
274         and overwrites), this writes the minimal profile-driven LIGHT bundle and ? by
default ? **never
275         overwrites an existing file**, so a user's edits or the first-run wizard's choices
are preserved
276         (idempotent first-run seeding). Pass ``overwrite=True`` to force. Creates parent
dirs.
277         """
278         cfg_path = Path(path) if path is not None else default_config_path()
279         if cfg_path.exists() and not overwrite:
280             return cfg_path, False
281         cfg_path.parent.mkdir(parents=True, exist_ok=True)
282         cfg_path.write_text(
283             json.dumps(LIGHT_DEFAULT_CONFIG, indent=2) + "\n", encoding="utf-8"
284         )
285         return cfg_path, True
286
287
288     # Backwards-compatible alias used by some older call sites during transition.
289     def get_default_config() -> dict[str, Any]:
290         """Return defaults as a plain dict (for transitional use only)."""
291         return _get_builtin_defaults()
292
```

15. user (2026-07-02T03:01:42.846Z)

```
No matches found
```

16. user (2026-07-02T03:02:27.666Z)

Structured output provided successfully